

WHAT DOES RETRIEVED MEMORY BUY? ACCURACY, EVIDENCE AND WRITE COST

Anonymous authors

Paper under double-blind review

ABSTRACT

This empirical systems study investigates how file-based project memory and hybrid retrieval compare in answer accuracy and total model cost under equal context-character budgets. We evaluate two memory rails using paired question-answering episodes drawn from an adapted LongMemEval single-session-user slice, executed by the qwen-plus model. The baseline writes facts to a disk file and truncates its prompt prefix, while the enhanced arm scores process-local working memory entries using lexical similarity combined with temporal decay, access frequency, and importance before packing them greedily. Under a restricted budget of 600 characters across 20 questions, the baseline achieved an accuracy of 0.0%, whereas the enhanced arm reached 65.0%. However, this accuracy gain required a total token ratio of 21.70 when accounting for memory-writing conversations. Expanding the budget to 2400 characters raised baseline accuracy to 60.0% and enhanced accuracy to 95.0%, with a total token ratio of 15.64. These findings expose a concrete tradeoff: hybrid selection improves evidence recall and answer correctness under tight constraints, but the model-backed memory construction imposes substantial write costs that must be amortized over subsequent queries. The study is limited to process-local memory without cross-restart persistence or team synchronization.

1 INTRODUCTION

Language models operating as conversational agents require mechanisms to retain user-specific facts across interactions. File-based project memory offers a straightforward approach by appending facts to a persistent document, but it suffers from rigid truncation when context windows are constrained. Hybrid retrieval systems attempt to overcome this limitation by dynamically selecting relevant memories based on query similarity and recency. This study addresses a specific operational question: How do file-based project memory and hybrid retrieval compare in answer accuracy and total model cost under equal context-character budgets?

Existing memory architectures often report retrieval improvements without fully accounting for the computational expense of constructing those memories. When a system uses model-backed conversations to write facts into working memory, the token consumption of those writing operations becomes part of the total cost. Evaluating only the query phase obscures the true resource requirements of maintaining an active memory store. Furthermore, comparing a truncated file prefix against a scored selection introduces format overhead confounds that must be explicitly acknowledged rather than treated as isolated retrieval gains.

This manuscript presents a small empirical study of a current implementation, not a novel proven multi-level memory system. The evaluation relies on paired observations where both arms receive identical facts, questions, and model configurations, differing only in their memory source and write mechanism. By alternating arm order across questions and verifying isolated query histories, the protocol controls for sequence effects within the limits of a single-run design.

Three contributions emerge from this auditable systems analysis. First, we quantify answer quality under equal full-section character budgets, revealing how format overhead disadvantages the baseline at smaller sizes. Second, we examine the association between complete evidence survival and correct answers, showing that partial facts can still yield correct responses when budgets expand.

Third, we measure total write-plus-query cost, demonstrating that memory construction expenses fundamentally alter the efficiency profile of hybrid retrieval compared to simple file storage.

The scope is deliberately narrow. Both budgets use independent questions from an adapted LongMemEval-S single-session-user slice, converting histories into fixed fact episodes with target answer turns and distractor turns. This adaptation does not constitute an official benchmark score. No offline retrieval ablation has been performed; the observed differences reflect whole-system behavior including budget sensitivity, whole-item packing, and format overhead rather than isolated component contributions.

2 RELATED WORK

Research on memory for language models spans organizational architectures, evidence access strategies, and evaluation methodologies. Understanding these themes clarifies the position of the current file-based and hybrid approaches within the broader landscape.

Memory control and organization systems define how information is structured and retained over time. MemGPT proposes virtual context management inspired by operating-system hierarchies, moving information across memory tiers to support document analysis and multi-session chat (Packer et al., 2023). The enhanced rail evaluated here draws conceptual inspiration from such tiered designs but implements only process-local L1 memory without autonomous virtual-memory controllers or cross-restart persistence. MemoryBank supports retrieval and updates for sustained personalized interaction, incorporating forgetting and reinforcement influenced by elapsed time and importance (Zhong et al., 2023). While the current temporal ranking heuristic shares surface similarities with elapsed-time weighting, it does not replicate the companionship evaluation or dynamic reinforcement mechanisms described in that work. A-MEM creates structured notes and dynamically links and evolves memories, drawing on Zettelkasten organization principles (Xu et al., 2025). In contrast, the fixed lexical ranking used here lacks autonomous linking and evolution capabilities, and no empirical comparison to A-MEM was conducted.

Evidence access mechanisms determine how stored information reaches the model during inference. Generative Agents synthesize natural-language experience records into reflections and dynamically retrieve them for planning behavior in interactive simulations (Park et al., 2023). This study neither implements reflection nor tests social behavior; it distinguishes basic record retrieval from that broader agentic architecture. The positioning of retrieved information also affects utility. Research on long contexts demonstrates that relevant information position influences multi-document question answering and key-value retrieval, with middle positions often proving harder to utilize (Liu et al., 2023). This motivates attention to information access patterns in the current experiments, though it does not establish that the observed prefix truncation in the baseline is caused by the same positional phenomenon.

Evaluation frameworks provide the metrics necessary to audit memory systems. LongMemEval evaluates extraction, multi-session reasoning, temporal reasoning, updates, and abstention by separating indexing, retrieval, and reading phases (Wu et al., 2024). The submitted adaptation covers only a narrow single-session-user slice of this broader benchmark, utilizing its fact-episode structure without claiming official task coverage. Together, these sources highlight that memory systems involve intertwined organizational, access, and evaluation challenges, motivating the paired cost-accuracy analysis presented here.

3 METHOD

The experimental method compares two memory rails operating under identical character budgets, model identities, and evaluation criteria. The core operational difference lies in how each arm stores and retrieves facts prior to answering a question. Both arms receive the same facts, questions, system prompts, and no external tools. The methodology schematic illustrates the flow from memory scoring through budget-constrained packing, which the reader should examine to understand that scoring precedes selection rather than following it Figure 1. The parallel arm comparison diagram further clarifies how the two paths maintain separate prompts and memory sources while sharing the character budget constraint Figure 2.

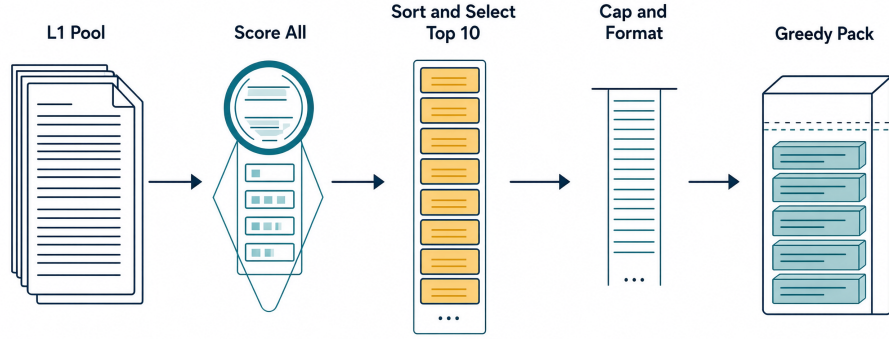


Figure 1: Conceptual schematic of the enhanced memory rail pipeline. All L1 entries receive a composite score combining lexical similarity, time decay, access frequency and importance. Entries are sorted descending and the top ten are considered. Each candidate is capped at five hundred characters, formatted, then greedily packed as a whole item under the shared section character budget. Smaller items may fill gaps left by skipped oversized ones.

3.1 FILE-BASED PROJECT MEMORY

The baseline represents the actual ProjectMemoryRail implementation. Facts are written directly to an isolated JIUWENSWARM.md file on disk. During the query phase, the prompt includes file metadata and is truncated to fit the full-section character budget. This truncation takes a prefix of the file content, meaning that facts appearing later in the document may be excluded entirely when the budget is tight. The baseline disk writing incurs no model-token charge, although this does not imply zero CPU, disk, or general resource cost. Crucially, this baseline is not a retrieval-augmented embedding system; it relies solely on sequential file reading up to the character limit. The inclusion of file metadata consumes part of the shared character budget, creating a format overhead confound that disadvantages the baseline relative to the enhanced arm, particularly at smaller budget sizes.

3.2 HYBRID SELECTION FROM WORKING MEMORY

The EnhancedMemoryRail writes completed user-assistant exchanges to process-local L1 memory. Rather than employing learned embeddings, the experiment uses a composite scoring formula combining lexical Jaccard similarity over lowercase regex word tokens with temporal decay, capped log access frequency, and an importance weight. The architecture defines L2 and L3 classes, but this specific Rail does not route writes to those tiers, persist them across process restarts, or synchronize a team. MemGPT serves as conceptual context for tiered memory management rather than an implemented or tested baseline (Packer et al., 2023). The retrieval engine scores all supplied L1 memories, sorts them in descending order, and returns the top candidates. Selected memories are then assembled under the same full-section character budget as the baseline. Each displayed exchange undergoes a preliminary content cap before formatting, and the system greedily appends the whole formatted item if it fits alongside section headers; otherwise, it skips the item and continues evaluating smaller candidates. This whole-item packing means that original exchange integrity is not guaranteed due to the preliminary cap.

3.3 ISOLATION AND COST ACCOUNTING

Each public-data question receives its own fact episode and fresh agent instances to prevent cross-contamination. The baseline writes facts to disk without model interaction, while the enhanced arm

performs model-backed memory-writing conversations that request acknowledgments and store both user prompts and assistant responses. The formal question then uses a new conversation ID to ensure isolation. The runner verifies isolated query history and actual memory injection, alternates arm order across questions to mitigate sequence effects, and records both query usage and memory-write usage (Wu et al., 2024). Latency measurements include write operations and are amortized over paired observations. Correctness is determined via accepted-answer string matching, which requires semantic review for edge cases. Evidence recall demands that the complete target fact survive prompt truncation or packing to be counted as recovered. The protocol produces two separate prompts and two isolated model calls per question; the arms share the assigned budget value and model identity but never merge prompt content or conversation history.

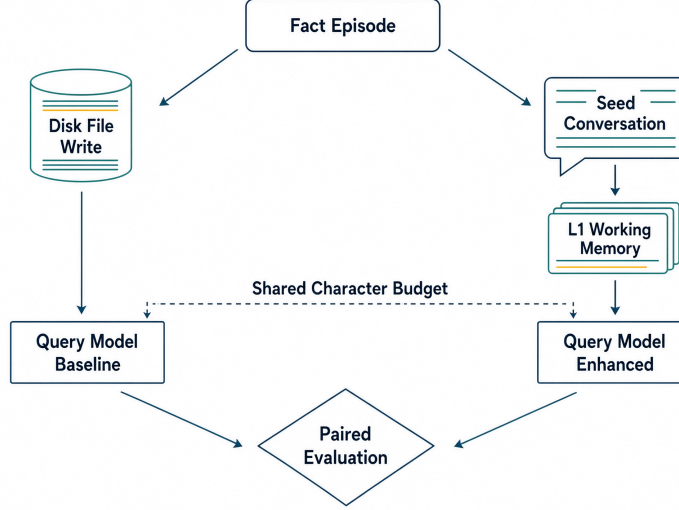


Figure 2: Conceptual schematic of the paired experimental arms. The baseline writes facts directly to disk without model tokens, while the enhanced arm constructs memory via model-backed seed conversations stored in process-local L1. Both query models operate under an identical full-section character budget but produce entirely separate prompts and isolated calls. Evaluation uses accepted-answer string matching and evidence recall over paired observations.

3.4 BUDGET AND COST FORMALIZATION

Let q be a question, M its episode memory, and B the cap on the entire injected memory section, including headings and metadata. The baseline constructs a file-derived section and keeps its prefix. The retrieved arm scores stored exchanges, takes at most $k = 10$ candidates, and greedily appends complete formatted items that fit. An over-budget item is skipped rather than partially appended; later smaller items can still be considered. Thus equal B does not imply equal usable fact content or identical prompt formatting.

$$s(q, m) = 0.4J(T(q), T(m)) + 0.32^{-a(m)/h} + 0.2 \min\left\{\frac{\log(1 + f(m))}{\log(101)}, 1\right\} + 0.1I(m), \quad (1)$$

where T lowercases text and keeps alphanumeric/underscore word tokens longer than one character; $J(A, C) = |A \cap C| / |A \cup C|$ (zero for an empty set); $a(m)$ is age in seconds; $h = 86400$ is the decay half-life; $f(m)$ is the recorded access count; and $I(m)$ is a bounded heuristic importance score. These are the implementation’s default weights, not learned or tuned coefficients. No embedding model is invoked. L1 holds at most 20 exchanges with a one-hour time-to-live; individual displayed exchanges are capped at 500 characters before packing.

The cost estimand includes initialization through model-backed memory writes:

$$C_g = \frac{\sum_{r \in \mathcal{Q}_g} u_r + \sum_{w \in \mathcal{W}_g} u_w}{N_{\text{valid}}}, \quad L_g = \frac{\sum_{r \in \mathcal{Q}_g} t_r + \sum_{w \in \mathcal{W}_g} t_w}{N_{\text{valid}}}. \quad (2)$$

Here u and t denote recorded token usage and elapsed time, respectively; $\mathcal{Q}_g$ and $\mathcal{W}_g$ include all recorded query and write attempts. The denominator counts valid matched question pairs. These timing sums exclude unrecorded process initialization and are not a full application wall-clock time. Missing usage remains unknown rather than zero.

For each valid pair, let $y_{g,i}$ indicate an accepted-answer substring match and $e_{g,i}$ indicate that the complete target fact survived in the injected text:

$$A_g = \frac{1}{N} \sum_i y_{g,i}, \quad R_g = \frac{1}{N} \sum_i e_{g,i}, \quad \Delta A = A_{\text{retrieved}} - A_{\text{file}}. \quad (3)$$

The two indicators differ: a truncated fact can contain a sufficient answer span. Wilson intervals describe finite-question uncertainty only. They do not estimate model-run variability, correct the selected slice’s bias, or establish significance.

Table 1: Audited execution protocol. No component ablation or weight tuning is claimed.

Step	Operation
Prepare	Create fresh isolated agents; give both arms the same fact episode.
Construct	Write the baseline file; seed retrieved L1 through separate fact conversations.
Query	Use a fresh conversation, no tools, one model iteration, and the assigned section budget.
Audit	Check actual prompt injection and isolated history before accepting a pair.
Aggregate	Match question IDs; count writes once per run; preserve excluded attempts in costs.

4 EXPERIMENTAL SETUP

The experiments evaluate the two memory rails across two distinct character budgets using the qwen-plus model. All questions originate from an adapted LongMemEval-S single-session-user slice, converted into fixed fact episodes containing a target answer turn alongside distractor turns. This adaptation provides controlled test cases but does not represent an official LongMemEval benchmark score. Each budget condition processes the same set of independent questions, allowing paired comparisons between the baseline and enhanced arms. Because repeated questions and budgets share a dataset, they are not independent samples, and no statistical significance tests are claimed. The evaluation focuses on four primary metrics: answer accuracy via string matching, complete evidence recall requiring full target fact survival, total tokens including both memory writes and query attempts amortized over valid paired observations, and latency encompassing the full execution pipeline.

4.1 RESTRICTED CONTEXT BUDGET

The restricted context budget allocates 600 characters for the full-section prompt. This tight constraint forces aggressive truncation in the baseline and selective packing in the enhanced arm. The experiment processes 20 unique questions with 1 repeat per question, yielding 20 paired observations after excluding 0 records from the baseline and 0 from the enhanced arm. Answer accuracy measures whether the model output matches the accepted answer string. Complete evidence recall verifies that the entire target fact survived the budget constraint. Total tokens account for all seed and query tokens consumed across both arms. Latency captures the wall-clock time from initiation to response generation, including memory construction for the enhanced path. The small budget inherently disadvantages the baseline because file metadata consumes a larger fraction of the available characters, leaving less room for actual fact content.

4.2 EXPANDED CONTEXT BUDGET

The expanded context budget increases the allocation to 2400 characters, providing substantially more room for fact retention. This condition evaluates the same 20 unique questions with 1 repeat, producing 20 paired observations with 0 baseline exclusions and 0 enhanced exclusions. The larger

budget reduces the relative impact of file metadata overhead on the baseline, allowing more facts to survive prefix truncation. Metrics remain identical to the restricted budget: answer accuracy, complete evidence recall, total tokens including writes, and end-to-end latency. The expanded budget tests whether the enhanced arm maintains its accuracy advantage when the baseline is less severely constrained by format overhead, and whether the cost differential changes when more memory items can be packed or read.

5 RESULTS AND ANALYSIS

The paired outcomes reveal distinct performance profiles across the two budget conditions. Wilson intervals reported here describe question-sampling uncertainty rather than repeated-run variability, as each configuration was executed once per budget. The analysis examines answer accuracy, evidence recall, token consumption, and latency for both arms, alongside paired outcome breakdowns that show where the systems agree or diverge.

5.1 RESTRICTED CONTEXT BUDGET

Under the restricted budget of 600 characters, the baseline achieved an answer accuracy of 0.0% with a Wilson interval ranging from 0.0% to 16.1%. The enhanced arm reached an accuracy of 65.0%, with an interval from 43.3% to 81.9%, representing an accuracy gain of 65 percentage points. Complete evidence recall mirrored these outcomes exactly: the baseline recorded 0.0% while the enhanced arm achieved 65.0%. The paired breakdown shows that both arms were correct on 0 questions, the enhanced arm alone was correct on 13 questions, the baseline alone succeeded on 0, and both failed on 7. No questions were answered correctly without full evidence in either arm, and no questions were answered incorrectly despite having full evidence. Token consumption differed drastically. The baseline consumed 4686 total tokens, averaging 234.30 per question, with zero seed tokens and zero seed model calls. The enhanced arm consumed 101669 total tokens, averaging 5083.45 per question, driven by 97805 seed tokens across 400 memory-writing calls. The total token ratio between enhanced and baseline was 21.70. For the enhanced arm, the cost per correct answer was 7820.69; the baseline produced no correct answers, making its cost-per-correct undefined. Query latency averaged 408.03 milliseconds for the baseline and 503.76 milliseconds for the enhanced arm. End-to-end latency, which includes memory writing, was 408.03 milliseconds for the baseline and 7375.67 milliseconds for the enhanced arm. The observed records associate failed answers with missing complete target evidence. They do not expose candidate scores or packing decisions, so they cannot distinguish ranking, truncation and packing as causes of individual failures.

5.2 EXPANDED CONTEXT BUDGET

Expanding the budget to 2400 characters altered the performance landscape. Baseline accuracy rose to 60.0% with a Wilson interval from 38.7% to 78.1%. Enhanced accuracy reached 95.0%, with an interval from 76.4% to 99.1%, yielding an accuracy gain of 35 percentage points. Evidence recall showed a divergence from accuracy for the baseline: complete evidence recall was 55.0%, lower than its answer accuracy. This occurs because a partial fact surviving truncation can still contain sufficient information for a correct answer. The enhanced arm maintained parity between accuracy and evidence recall at 95.0%. Paired outcomes shifted accordingly: both arms were correct on 12 questions, enhanced alone on 7, baseline alone on 0, and both wrong on 1. The baseline produced 1 correct answer without full evidence, confirming the partial-fact phenomenon. Neither arm produced wrong answers when full evidence was present. Token costs remained heavily skewed toward the enhanced arm. The baseline consumed 16877 total tokens, averaging 843.85 per question, with a cost per correct answer of 1406.42. The enhanced arm consumed 264036 total tokens, averaging 13201.80 per question, with 249398 seed tokens across 400 calls and a cost per correct answer of 13896.63. The total token ratio was 15.64, and the cost-per-correct ratio was 9.88. Query latency averaged 717.01 milliseconds for the baseline and 601.41 milliseconds for the enhanced arm, showing the enhanced query phase was faster despite higher overall end-to-end latency of 7133.41 milliseconds versus 717.01 milliseconds for the baseline. The diagnostic cases show variation in answer correctness and complete evidence survival. Candidate ranks and item-level packing traces

were not recorded; the larger-budget observations therefore do not establish which selection step caused a particular outcome.

Table 2: Restricted Context Budget. Costs include memory writes; missing usage is not treated as zero.

Metric	Baseline	Enhanced
Correct answers	0	13
Paired observations	20	20
Answer accuracy	0.0%	65.0%
Evidence recall	0.0%	65.0%
Query tokens (total)	4686	3864
Write tokens (total)	0	97805
Total tokens / paired observation	234.30	5083.45
Mean query latency (ms)	408.03	503.76
Amortized total latency (ms)	408.03	7375.67
Excluded records	0	0

Table 3: Expanded Context Budget. Costs include memory writes; missing usage is not treated as zero.

Metric	Baseline	Enhanced
Correct answers	12	19
Paired observations	20	20
Answer accuracy	60.0%	95.0%
Evidence recall	55.0%	95.0%
Query tokens (total)	16877	14638
Write tokens (total)	0	249398
Total tokens / paired observation	843.85	13201.80
Mean query latency (ms)	717.01	601.41
Amortized total latency (ms)	717.01	7133.41
Excluded records	0	0

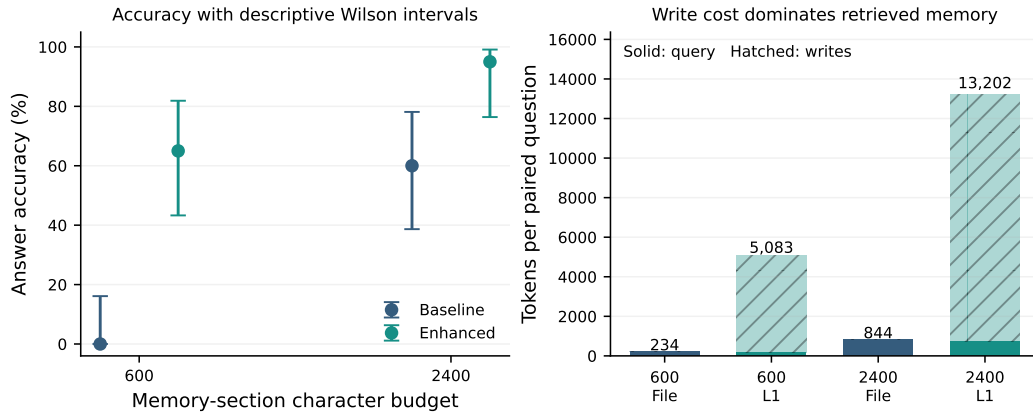


Figure 3: Quality and resource tradeoff. Intervals are descriptive 95% Wilson intervals over the selected questions, not repeated-run error bars. Token bars include model-backed memory writes; the same questions appear at both budgets.

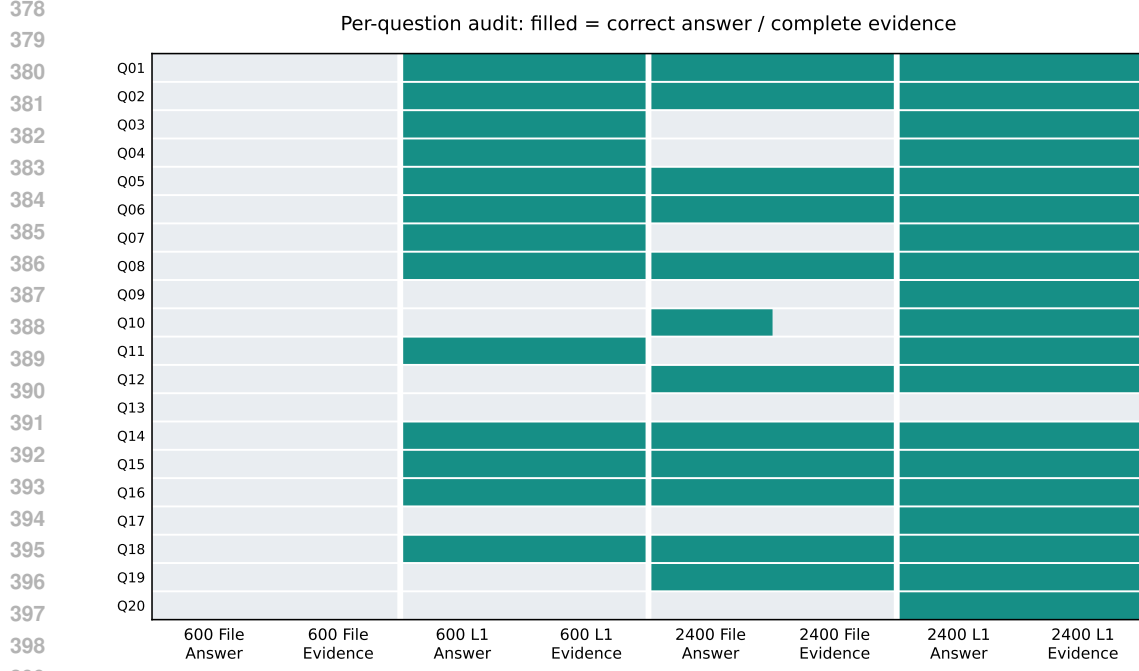


Figure 4: Matched question-level correctness and full-evidence recall across both budgets. Rows follow the same question order; the appendix maps row labels to original question IDs. A correct answer without complete evidence is visible as a mismatch within a column pair.

6 DISCUSSION AND LIMITATIONS

The results expose a fundamental engineering tradeoff between memory construction cost and query-phase accuracy. Hybrid retrieval improves answer correctness, particularly under tight character budgets where file-based truncation eliminates target facts entirely. However, this improvement requires substantial model-backed memory writing, inflating total token consumption by ratios exceeding fifteen to one even at larger budgets. Practitioners deploying such systems must amortize these write costs over many subsequent queries to justify the initial investment. If an agent interacts infrequently, the per-question token burden of memory construction may outweigh the accuracy benefit.

Several confounders limit causal interpretation. The smaller budget inherently disadvantages the baseline because file metadata consumes characters that could otherwise hold facts. This format overhead confound means the observed accuracy gap at 600 characters reflects both retrieval strategy and structural inefficiency. At the larger budget, the baseline partially recovers, yet the enhanced arm still maintains an advantage through targeted selection rather than sequential reading. Whole-item packing introduces another variable: the preliminary content cap applied before greedy insertion can truncate exchanges, meaning the enhanced arm does not guarantee original exchange integrity even when an item is selected. Distinguishing budget sensitivity from component ablation remains impossible without offline retrieval testing, which was not performed in this study.

Missing measurements constrain the scope of conclusions. No offline retrieval ablation isolates the contribution of lexical scoring versus temporal decay versus frequency weighting. The string-based accepted-answer matching requires semantic review for borderline cases, and context metadata consumption varies between arms. The current enhanced Rail writes only to process-local L1 memory; L2 and L3 persistence mechanisms and team synchronization features defined in the architecture remain untested. These experiments do not validate any automatic paper-generation capability; they audit a specific memory question-answering pipeline. Furthermore, the adapted LongMemEval slice covers only single-session-user interactions, leaving multi-session reasoning, temporal updates, and abstention behaviors unevaluated.

The observation that baseline answer accuracy exceeds complete evidence recall at the larger budget carries practical implications. It demonstrates that models can extract correct answers from partial context fragments, suggesting that strict evidence recall thresholds may overstate failure rates in file-based systems. Conversely, the enhanced arm maintained perfect alignment between evidence recall and accuracy, indicating that its scoring mechanism either retrieves complete facts or fails entirely, without producing partial survivals that happen to contain answers. This binary behavior simplifies debugging but may reduce robustness when exact fact boundaries are ambiguous.

Latency patterns reveal that query-phase execution can be faster for the enhanced arm at larger budgets, likely because the packed context is more concise than the raw file prefix. However, end-to-end latency remains dominated by memory-writing conversations. Engineers optimizing for interactive response times must separate query latency from construction latency, recognizing that total token cost including memory writes represents the true operational expense. The absence of repeated-run error bars means these observations describe a single execution trace rather than a stable distribution, reinforcing the need for cautious interpretation.

7 CONCLUSION

This auditable systems study examined how file-based project memory and hybrid retrieval compare in answer accuracy and total model cost under equal context-character budgets. Using paired observations from an adapted LongMemEval single-session-user slice processed by qwen-plus, the analysis confirms that hybrid selection from working memory improves answer accuracy and evidence recall relative to truncated file storage. Under a restricted budget of 600 characters, the enhanced arm achieved an accuracy gain of 65 percentage points over the baseline, while under an expanded budget of 2400 characters, the gain was 35 percentage points. However, these accuracy improvements carry substantial cost: total token consumption including memory writes exceeded the baseline by ratios of 21.70 and 15.64 for the small and large budgets respectively. The findings demonstrate that retrieved memory buys measurable accuracy, particularly when character constraints force aggressive file truncation, but the model-backed construction required to populate that memory imposes token costs that must be carefully amortized in production deployments. Future work should isolate budget sensitivity from retrieval components through offline ablations and extend evaluation to multi-session scenarios beyond the current single-session scope.

REFERENCES

- Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the Middle: How Language Models Use Long Contexts. *arXiv preprint*, 2023. URL <https://arxiv.org/abs/2307.03172v3>.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as Operating Systems. *arXiv preprint*, 2023. URL <https://arxiv.org/abs/2310.08560v2>.
- Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative Agents: Interactive Simulacra of Human Behavior. *arXiv preprint*, 2023. URL <https://arxiv.org/abs/2304.03442v2>.
- Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. *arXiv preprint*, 2024. URL <https://arxiv.org/abs/2410.10813v2>.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic Memory for LLM Agents. *arXiv preprint*, 2025. URL <https://arxiv.org/abs/2502.12110v11>.
- Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. MemoryBank: Enhancing Large Language Models with Long-Term Memory. *arXiv preprint*, 2023. URL <https://arxiv.org/abs/2305.10250v3>.

A QUESTION-LEVEL AUDIT AND REPRODUCIBILITY

The following identifiers link directly to the submitted raw records. A and E denote answer correctness and complete-evidence recall. These are recorded single-run outcomes, not independent replications across budgets.

Table 4: Question audit at the 600-character budget.

Row	Original question ID	File A	File E	L1 A	L1 E
Q01	0862e8bf	0	0	1	1
Q02	118b2229	0	0	1	1
Q03	15745da0	0	0	1	1
Q04	19b5f2b3	0	0	1	1
Q05	1faac195	0	0	1	1
Q06	36580ce8	0	0	1	1
Q07	3d86fd0a	0	0	1	1
Q08	545bd2b5	0	0	1	1
Q09	58bf7951	0	0	0	0
Q10	60d45044	0	0	0	0
Q11	66f24dbb	0	0	1	1
Q12	6ade9755	0	0	0	0
Q13	75499fd8	0	0	0	0
Q14	86f00804	0	0	1	1
Q15	8a137a7f	0	0	1	1
Q16	8ebdbe50	0	0	1	1
Q17	c14c00dd	0	0	0	0
Q18	c8c3f81d	0	0	1	1
Q19	caf9ead2	0	0	0	0
Q20	ccb36322	0	0	0	0

Table 5: Question audit at the 2400-character budget.

Row	Original question ID	File A	File E	L1 A	L1 E
Q01	0862e8bf	1	1	1	1
Q02	118b2229	1	1	1	1
Q03	15745da0	0	0	1	1
Q04	19b5f2b3	0	0	1	1
Q05	1faac195	1	1	1	1
Q06	36580ce8	1	1	1	1
Q07	3d86fd0a	0	0	1	1
Q08	545bd2b5	1	1	1	1
Q09	58bf7951	0	0	1	1
Q10	60d45044	1	0	1	1
Q11	66f24dbb	0	0	1	1
Q12	6ade9755	1	1	1	1
Q13	75499fd8	0	0	0	0
Q14	86f00804	1	1	1	1
Q15	8a137a7f	1	1	1	1
Q16	8ebdbe50	1	1	1	1
Q17	c14c00dd	0	0	1	1
Q18	c8c3f81d	1	1	1	1
Q19	caf9ead2	1	1	1	1
Q20	ccb36322	0	0	1	1

B OBSERVED DIAGNOSTIC CASES

Cases below are selected deterministically for diagnostic categories, not as a random qualitative sample. Answers are verbatim recorded outputs; questions are copied from the adapted dataset.

Retrieved-Only Success. Budget 600; question 0862e8bf.

Question: What is the name of my cat?

Accepted answer: Luna

File answer: UNKNOWN

Retrieved answer: Your cat's name is Luna.

Retrieval Failure. Budget 600; question 58bf7951.

Question: What play did I attend at the local community theater?

Accepted answer: The Glass Menagerie

File answer: UNKNOWN

Retrieved answer: UNKNOWN

Partial Evidence. Budget 2400; question 60d45044.

Question: What type of rice is my favorite?

Accepted answer: Japanese short-grain rice

File answer: Your favorite type of rice is Japanese short-grain rice.

Retrieved answer: Your favorite type of rice is Japanese short-grain rice.

C ARTIFACT AND GENERATION PROVENANCE

The submission includes the adapted data, original write/query records, normalization code, citation metadata and a hashed source snapshot. Writing assistance uses a separate model from the experimental query model. Its actual requests, generated text and usage are saved separately; none of those calls contributes to the experimental accuracy. The conceptual diagrams are AI-generated from source-linked method specifications. Their design contracts, prompts and declared review status are included; human scientific review remains required. All empirical plots and tables are rendered programmatically from the recorded data.